

```
~/sdairs git:(main) $ cat posts/there-is-no-data-engineering-roadmap.md
```

There is no Data Engineering roadmap

Between Reddit, twitter, LinkedIn and various Slack communities, I see multiple junior folk looking to break into Data Engineering and asking for advice. Every single day. Many ask for a “roadmap” or some kind of step by step lesson plan that will land them their dream job. I don’t believe that such a roadmap exists.

Newbies are welcome 🙌

I have seen some say “Data Engineering is not an entry level role” and this is nothing more than toxic gatekeeping. Data Engineering is no more, and no less, complex than any other software discipline. Every discipline is open to newbies. If you want to get into data, you can do it. You don’t *need* to “graduate” into it from a different discipline.

Moving sideways into Data Engineering is very common, not because it’s necessary, but because Data Engineering is relatively new as a somewhat well-defined job category. Data Engineering teams haven’t been commonplace for long, in fact, there’s many industries still just starting to catch on. Many Data Analysts and Software Engineers already have at least *some* level of hands-on experience with data, so it makes total sense to use & develop those skills. This has happened with every single kind of

engineering role in the history of engineering. But there's only so many people who want to make the switch, and you can't reallocate *everyone* from your other teams. So, Data Engineering absolutely needs entry-level engineers.

People used to say "Software Engineering is not an entry level role". They don't anymore, because people know it is total rubbish.

Everyone is welcome in data.

All you need is ~~love~~ SQL

So, how does an entry-level engineer get started in Data Engineering?

Firstly, go unfollow all those influencers on LinkedIn and Twitter. You don't need them, in fact, they are dangerous. They're not here to guide, help or teach you. They will take you down a path of failure so that you are more open to giving them your money for a quick win ([rant](#)).

With that out the way, understand that there is no roadmap. There is no single path, no clear linear progression of knowledge. No one can tell you that you absolutely must learn A, then B, then C and you're guaranteed to be a successful Data Engineer.

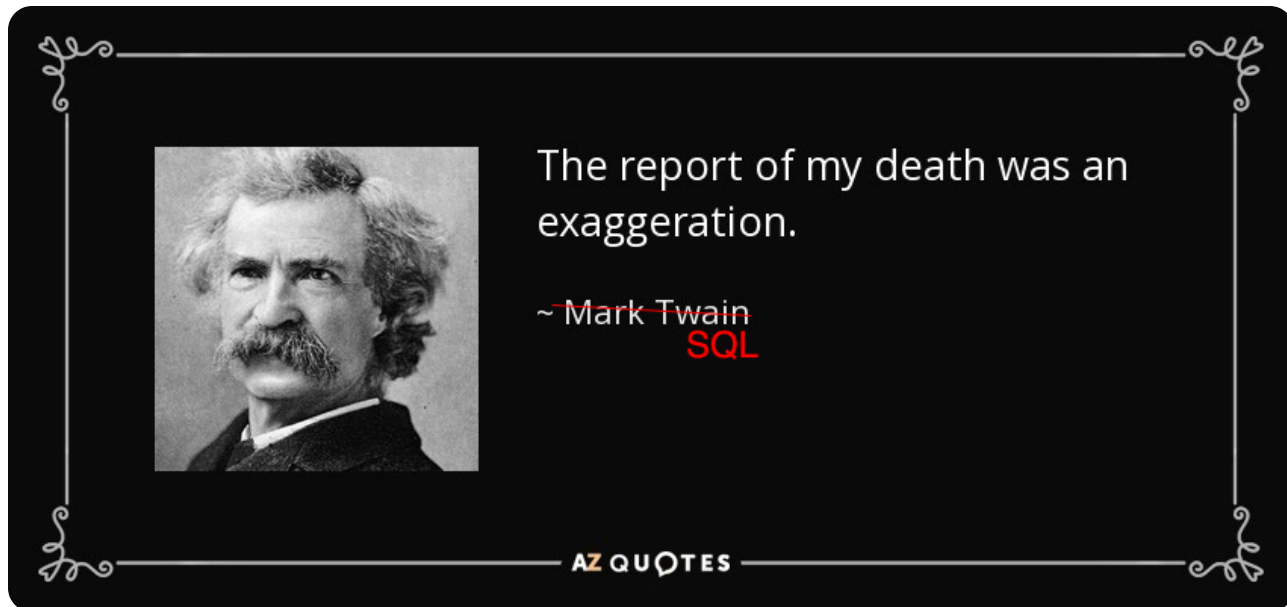
The same applies to pretty much all engineering roles; front end, back end, embedded systems, networking, analytics. Whatever.

In all of these cases, there are basics that *everyone* should get familiar with, and these are usually enough to get you your first gig. Remember, there is a fundamental difference between “What do I need to get my first job?” and “How do I progress my career?”.

For Data Engineering, there is only one skill that is absolutely, non-negotiably, the first thing you should learn to get started.

SQL.

Yep, SQL. It's not dead. It never will be. SQL is the cockroach of data and it's not going anywhere. People have tried to displace it, and they have all failed.



SQL is the only skill that every single Data Engineer uses every single day. No other skill or tool can claim the same. Python is common, some folks are using Scala, Snowflake is popular...but there's more data teams *not* using those tools than those who *are*. But not for SQL.

You're an entry level engineer, you don't need to be an expert in SQL. You need to be able to solve problems. When you write some shit SQL, and you absolutely will, one of two things will happen:

Thing 1, someone will tell you it's shit and you'll learn how to do it better.

Thing 2, people will thank you for solving the problem and ask you to do something else.

Win win.

How should I learn SQL?

If you're new to SQL and databases, you should know that "SQL" is very poorly standardised. You'll hear folks say "ANSI SQL" which many think is some kind of standard, but it's not really. Anyway, if that topic interests you, [read up on it](#)^[↗]. There is a common SQL base, but pretty much every single database in existence customises and extends SQL to do whatever it wants.

This means that SQL you write for Postgres *might* work in MySQL, but don't be surprised if it doesn't. The same is true across Microsoft's SQL Server, Oracle Database, BigQuery, Redshift, Snowflake, ClickHouse and any other database you can think of. The different flavours of SQL used by these databases are called "dialects".

This can make it daunting to get started, but it's no different than getting into Software Engineering. There's a million

programming languages and most people start with one and then try a bunch of others.

So, pick any database, don't worry about the dialect. The database is simply a vehicle for you to learn SQL.

Stuck? Start with [Postgres](#). It's the world's favorite free, open source database. You can't go wrong starting with Postgres.

Follow some simple tutorials; get it set up, load some data and start asking questions with SQL.

Google around for "SQL challenges", there's loads. Some are better than others, just go through them all and challenge yourself. As your knowledge improves, look for harder problems and bigger data sets.

When you're starting to feel confident - change database. Try solving the same problems with [MariaDB](#). Then try out [Google's BigQuery](#) (there is a generous free-tier for BigQuery, be careful to stay under the limits and you won't pay anything).

Pay attention to how your queries change, particularly with more complex queries. Notice that different kinds of queries are faster or slower between databases. Get used to reading the SQL reference documentation for each of these databases.

If you need something more guided, there are plenty - literally *thousands* - of free SQL resources on the internet. There's nothing wrong with following a *free* SQL introduction course, but always challenge what you learn by applying it to a different database.

You will have time to specialise in a specific database later in your career, now is not the time.

If you're in the UK, the UK Gov is sponsoring a whole bunch of entry-level bootcamps across loads of sectors. One of the biggest areas of funding is Data Engineering. I can't vouch for these bootcamps, some look better than others, but they are *free*, so if you're eligible, why not? <https://www.gov.uk/guidance/find-a-skills-bootcamp/>^[↗]

What if I already know SQL?

If you're coming from a Data Analytics background, or any other role where you're already reasonably comfortable with SQL, then you have it easy. Anyone with this background should be able to start looking for an entry-level or junior Data Engineering role.

Now, if you're sitting on 15 years of experience as an Analyst, going back to a junior role might not be something you're willing to do - but that's a different discussion.

What's next?

What about Python? Pandas? dbt? Rust? Airflow? Spark?

Later. These are all things you can learn on the job *if the job even needs them*.

Go get your first data job. I'm not going to tell you it will be easy. Lots of people struggle to find the right entry-level job in all fields of engineering.

But when you land it, make it your primary goal to absorb the knowledge from your new colleagues. Learn something every single day.

When the learning stops, move on. Use what you've learnt to get a pay bump and find new people to learn from.

Rinse and repeat. That's your roadmap.

Everything else comes later. Go get your hands dirty.

A quick rant

Unfortunately, I see a lot of bad advice handed out. Now, much of it is others just innocently sharing an opinion, but, often there is a clear financial incentive behind it. Vendors who want their tools to be the "baseline" to enter the industry. So-called "influencers" who take money from those vendors, or want to convince you to buy their Data Engineering bootcamps. Because, if data engineering is spooky and complicated, you're more inclined to buy their "Land your first job in 90 days" course, right?

Now, there's nothing wrong with vendors advertising their tools, or individuals creating genuinely helpful content that earns themselves a living. Tools have a place in the world, as do creators. But certain bad actors target junior and entry-level engineers who don't have the experience to identify blatant bullshit from real advice. Be wary.

~/sdairs git:(main) \$



NORMAL | ~/sdairs | 23 posts | utf-8 | main

2026-06-06